

ACID

The 4 promises every reliable database makes you.



Atomicity

All or nothing



Consistency

Rules always hold



Isolation

No stepping on toes



Durability

Survives a crash

Swipe to master each one →



Atomicity

All or nothing

A transaction fully completes, or it fully rolls back. There is no halfway.

EXAMPLE · \$100 transfer, Ali → Sara

Debit Ali \$100 AND credit Sara \$100 are one unit.

If the credit fails, the debit is undone too — the money never vanishes.

Without it: Ali could lose \$100 that Sara never receives.



Consistency

Rules always hold

Every transaction moves the database from one valid state to another — all constraints stay satisfied.

EXAMPLE · \$100 transfer, Ali → Sara

Rule: total money in the bank never changes on a transfer.

Rule: no balance can go negative.

A transfer that breaks either rule is rejected, not saved.

Without it: Corrupt data slips in and quietly breaks everything downstream.



Isolation

No stepping on toes

**Concurrent transactions don't interfere.
The result is as if they ran one after another.**

EXAMPLE · \$100 transfer, Ali → Sara

Ali sends \$100 to Sara and to Zoya at the same instant.

Isolation stops them from both reading the old balance and letting Ali spend the same \$100 twice.

Without it: Race conditions let users double-spend or read half-finished data.



Durability

Survives a crash

Once a transaction is committed, it stays — even if the power dies one second later.

EXAMPLE · \$100 transfer, Ali → Sara

The app shows "Transfer complete" to Ali.

The server loses power immediately after.

On reboot, Sara still has the \$100. A commit is permanent.

Without it: "Saved" data disappears after a crash — users lose trust instantly.

ACID, in one line

Keep this for your next system-design interview.

A

Atomicity

All steps commit together, or none do.

C

Consistency

Every commit leaves the data valid.

I

Isolation

Concurrent txns don't corrupt each other.

D

Durability

Committed data survives any crash.

Relational DBs (PostgreSQL, MySQL) guarantee ACID. Many NoSQL stores trade it for scale — that model is called BASE. Know when each trade-off is worth it.

Found this useful? Repost to help a dev ♻️